

Compiler Design Laboratory

Introduction to Flex Tool

Objective

To learn the basics of the **Flex** tool and to use it for building a simple lexical analyzer.

1. Introduction to Flex

Flex (Fast Lexical Analyzer Generator) is a tool for automatically generating lexical analyzers (scanners). It takes a set of regular expressions as input and produces a C program that can recognize tokens in a given input stream.

Flex is widely used in compiler design to implement the **Lexical Analysis phase**.

2. Checking Availability of Flex on Ubuntu

To check whether Flex is installed on your Ubuntu machine, open the terminal and type:

```
flex --version
```

If Flex is installed, it will display the version number. Otherwise, it will show a command not found error.

3. Installing Flex on Ubuntu

If Flex is not installed, install it using the following commands:

```
sudo apt update
sudo apt install flex
```

Also make sure that gcc is installed:

```
gcc --version
```

4. Basic Structure of a Flex Program

A Flex program consists of three sections:

```
%{
    C declarations
%}
```

```

%%
    Regular expression rules and actions
%%

    User-defined C functions

```

- The first section contains C declarations and header files.
- The second section contains token patterns and corresponding actions.
- The third section contains supporting C code and functions.

5. Sample Flex Program

The following Flex program counts the number of identifiers and numbers in an input file.

```

%{
#include <stdio.h>
int id_count = 0;
int num_count = 0;
%}

%%
[a-zA-Z_][a-zA-Z0-9_]*    { id_count++; }
[0-9]+                    { num_count++; }
[ \t\n]+                  ;    /* Ignore whitespace */
.                          ;    /* Ignore other characters */
%%

int main() {
    yylex(); /* Calls the lexical analyzer to start scanning the
               input */
    printf("Number of identifiers = %d\n", id_count);
    printf("Number of numbers = %d\n", num_count);
    return 0;
}

```

6. How to Compile and Run a Flex Program

Follow these steps:

1. Save the program as `sample.1`

2. Generate C file:

```
flex sample.l
```

3. Compile the generated C file:

```
gcc lex.yy.c -o lexer -lfl
```

4. Run the program with input file:

```
./lexer < input.txt
```

7. Sample Input File

Create a file named `input.txt` with the following content:

```
int a = 10;  
float sum = a + 20;  
result = sum + 30;
```

8. Assignment for Students

Task 1

Write a Flex program to count the following from an input C program:

- Number of keywords
- Number of identifiers
- Number of operators
- Number of constants

Task 2

Modify the above Flex program to:

- Print each token along with its type
- Maintain a symbol table for identifiers

Task 3

Write a Flex program to remove all comments from a C program.

Submission Instructions

- Submit the Flex source code (.l file)
- Submit sample input file
- Submit output screenshots
- Include a short explanation of your program

Learning Outcome

After completing this experiment, students will be able to:

- Use Flex tool
- Write basic lexical analyzers
- Understand tokenization using regular expressions

Appendix: Important Flex Variables and Functions

The following variables and functions are frequently used in Flex programs and are essential for writing practical lexical analyzers.

1. yyin

`yyin` is a file pointer used by Flex to specify the input stream for the lexical analyzer. By default, Flex reads input from `stdin`. If we want to read input from a file, we can assign the file pointer to `yyin`.

```
/* yyin is used to change the input source of the lexer */  
FILE *fp = fopen("input.txt", "r");  
yyin = fp;      /* Now the lexer will read from input.txt instead  
                  of keyboard */
```

Purpose: To specify the input file from which the lexical analyzer should read.

2. yytext

`yytext` is a character array (string) that contains the **current lexeme** matched by the Flex pattern.

Whenever a pattern is matched, the corresponding lexeme is automatically stored in `yytext`.

```

/* yytext contains the currently matched token string */
[a-zA-Z_][a-zA-Z0-9_]*    {
    printf("Identifier found: %s\n", yytext);
}

```

Purpose: To access the actual text (lexeme) of the token that has just been recognized.

3. yywrap()

yywrap() is a function called by the lexical analyzer when the end of the input file is reached.

- If yywrap() returns 1, scanning stops.
- If yywrap() returns 0, scanning continues with another input file.

A simple definition of yywrap() is:

```

/* yywrap() tells the lexer what to do at end-of-file */
int yywrap() {
    return 1;    /* Return 1 means stop scanning */
}

```

Purpose: To inform the lexer what to do when end-of-file is reached.

Summary

Name	Purpose
yyin	Specifies the input file for the lexer
yytext	Holds the currently matched lexeme (token string)
yywrap()	Called at end-of-file to decide whether to continue or stop scanning